

Training the EMNIST DDPM

An explanatory packet for training/train.py

David Falk
APEX Laboratory, The University of Chicago

Abstract

This document walks through `training/train.py`, the script that trains a pixel-space, class-conditional DDPM on the EMNIST dataset using the `UNet2DModel` from `diffusers`. The wording follows the source-code comments closely, but is reorganised into prose with the relevant equations collected up front and the code shown only where it earns its place.

Contents

1	Overview	2
2	Configuration knobs	2
2.1	Size presets	3
3	Distributed setup	4
4	Data pipeline	4
4.1	Image transformations	4
4.2	Dataset and DataLoader	4
5	Model	5
5.1	What the U-Net is actually learning	5
5.2	The U-Net architecture, in three parts	5
5.3	Constructing the U-Net	6
5.4	The EMA model	7
5.5	Scheduler, optimizer, AMP	7
6	The forward diffusion process	8
7	Inside the training loop	9
7.1	Setting up the epoch	9
7.2	One mini-batch, end to end	9
7.2.1	CFG dropout	10
7.2.2	Sampling timesteps and noise	10
7.2.3	Forward pass and loss	11
7.2.4	Backprop with mixed precision and the optimizer step	11
7.2.5	EMA update	11
7.3	End of epoch: logging and checkpointing	12

1 Overview

The script trains a pixel-space class-conditional DDPM using `UNet2DModel` on the EMNIST dataset. The headline features are:

1. **Balanced split.** Equal samples per class, since the raw EMNIST class sizes are heavily skewed toward digits.
2. **Cosine beta schedule** (`squaredcos_cap_v2`) for better fine details.
3. **EMA weights** for smoother, more stable inference.
4. **Classifier-Free Guidance (CFG) dropout**, which enables guidance at generation time.

A small EMNIST quirk. The training pipeline merges letters whose upper- and lower-case forms are identical or highly similar (e.g. `s`, `c`) so that, intuitively speaking, the model does not try to learn/internalize some arbitrary difference between `s` and `S`. At inference time you can enter either `s` or `S`; the loader casts to the correct class label.

2 Configuration knobs

The top of `train.py` reads as a single configuration block. The most important entries are reproduced and explained below.

```
MODEL_NAME = "balanced_cfg_cosine_ema_600_steps"
SIZE_NAME = "large"
DATASET_SPLIT = "balanced"
EPOCHS = 100
LEARNING_RATE = 1e-4
STEPS = 600
BETA_SCHEDULE = "squaredcos_cap_v2"
EMA_DECAY = 0.999
CFG_DROPOUT_PROB = 0.10
DOWNLOAD_DATASET = False
```

MODEL_NAME

Used to construct the checkpoint filename.

SIZE_NAME

Key into the `SIZES` dictionary; controls image resolution, batch size, and channel widths.

DATASET_SPLIT

EMNIST split. `balanced` ensures equal samples per class so that no class dominates the loss.

EPOCHS

Number of full passes through the dataset.

LEARNING_RATE

AdamW step size.

STEPS

Number of diffusion timesteps; Gaussian noise is iteratively added to the original image over STEPS timesteps.

BETA_SCHEDULE

Each injection of Gaussian noise has variance β_t . This controls how β_t grows with t .

EMA_DECAY

Decay factor for the exponential moving average of the weights. Smooths the per-step fluctuations of the live model.

CFG_DROPOUT_PROB

Fraction of training samples whose label is replaced with the null label, enabling classifier-free guidance.

DOWNLOAD_DATASET

If True, EMNIST is downloaded on first run. Recommended: download it ahead of time.

2.1 Size presets

```
SIZES = {
    "small": (28, 2048, (64, 128, 256)),
    "medium": (64, 384, (64, 128, 256, 256)),
    "large": (96, 96, (96, 192, 384, 384)),
    "xl": (128, 32, (128, 256, 512, 512)),
}
```

Each entry expands into (img_res, batch_size, block_out_channels):

img_res

Final training image resolution; images are resized to `img_res`×`img_res`.

batch_size

Number of images processed per training step (per process).

block_out_channels

Number of feature channels at each U-Net resolution stage. One entry per downsampling stage.

The four resolution stages correspond to:

Stage	Resolution	#Channels	What it learns
1	<code>img_res</code> × <code>img_res</code>	<code>block_out_channels[0]</code>	low-level features (edges, strokes)
2	<code>(img_res/2)</code> × <code>(img_res/2)</code>	<code>block_out_channels[1]</code>	mid-level features (partial shapes)
3	<code>(img_res/4)</code> × <code>(img_res/4)</code>	<code>block_out_channels[2]</code>	higher-level structure (whole components)
4	<code>(img_res/8)</code> × <code>(img_res/8)</code>	<code>block_out_channels[3]</code>	bottleneck with global context for noise prediction

Feature maps in plain English. At each stage of the U-Net, the model creates multiple “feature maps”. Think of each feature map as shining a different “pattern revealer” laser beam across the image. Each beam slides over the entire image and lights up wherever its learned pattern appears. The brightness at each pixel answers: *“How strongly does this specific pattern exist here?”*

The number in `block_out_channels` is the number of different pattern revealers (feature maps) at that stage — e.g. 64 means 64 different learned pattern detectors, 256 means 256. As the U-Net

goes deeper, the spatial resolution shrinks and the number of feature maps grows: early stages detect simple local patterns, deeper stages detect more abstract structure. Larger numbers mean more pattern detectors, greater representational capacity, and more parameters/memory.

3 Distributed setup

Each GPU runs its own copy of the script. We coordinate them via `torch.distributed`:

```
rank = int(os.environ.get("RANK", "0"))
local_rank = int(os.environ.get("LOCAL_RANK", "0"))
torch.cuda.set_device(local_rank)
torch.distributed.init_process_group("nccl")
```

RANK

Global rank of this process. `RANK == 0` is the main process; logging and checkpointing are restricted to it.

LOCAL_RANK

Rank within the current machine; tells this process which GPU to use when a node has multiple GPUs.

nccl

NVIDIA Collective Communications Library — optimized for GPU-to-GPU communication and used to sync gradients across processes.

4 Data pipeline

4.1 Image transformations

Each EMNIST image is run through a deterministic `transforms.Compose` pipeline:

1. Resize the EMNIST image (28×28) to the target resolution (`img_res×img_res`).
2. Convert the image to a PyTorch tensor (integer pixel values become `float32` in `[0, 1]`).
3. Rotate 90° and flip along the height axis. EMNIST images are stored rotated and flipped — this undoes that, so they appear upright.
4. Scale from `[0, 1]` to `[-1, 1]` via $f(x) = 2x - 1$ (centres pixel values around 0).

4.2 Dataset and DataLoader

`datasets.EMNIST(...)` wraps the raw EMNIST files as a PyTorch `Dataset`. The two methods that matter at training time are:

- `__len__()` — how many samples exist.
- `__getitem__(i)` — the `(image, label)` tuple at index `i`, which gets called repeatedly during training.

A `DistributedSampler` decides “in what order should the dataset indices be read?” Normally PyTorch reads sequentially or shuffles, but in a distributed setup we have multiple processes and we must avoid having every GPU read the whole dataset and duplicate work / mess up gradients. The distributed sampler splits the indices across processes and keeps shuffling synchronised across processes when we call `sampler.set_epoch` later.

The `DataLoader` is then a batching and parallel-loading engine on top of the dataset/sampler:

- `batch_size = batch` — samples per training step per process.
- `num_workers = 8` — CPU subprocesses that load and preprocess in parallel.
- `pin_memory = True` — locks CPU memory pages for faster transfer to GPU.
- `drop_last = True` — if the dataset is not divisible by the batch size, drop the final smaller batch (otherwise gradient synchronisation across GPUs with different batch sizes is painful).

Finally, the number of valid class labels is `num_classes = len(dataset.classes)`, and we add one extra `NULL_CLASS` with index `num_classes` that will be used by CFG dropout.

5 Model

5.1 What the U-Net is actually learning

In diffusion training, we corrupt a clean image x_0 into x_t by gradually adding Gaussian noise over many timesteps. Training the model is just teaching it how to start with image x_t and remove noise.

Concretely, the model learns the function

$$\varepsilon_{\text{pred}} = f(x_t, t, \text{class_label})$$

and the training objective minimises

$$\text{MSE}(\varepsilon_{\text{pred}}, \varepsilon) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2,$$

where ε is the true Gaussian noise used to corrupt the image.

5.2 The U-Net architecture, in three parts

1) Downsampling path (the “compress” part). The image is gradually reduced in spatial size, e.g. $64 \times 64 \rightarrow 32 \times 32 \rightarrow 16 \times 16 \rightarrow 8 \times 8$. As the image gets smaller, the number of feature channels increases, and the model learns more abstract patterns at each step. Think of it as “zooming out to understand the overall structure”.

2) Bottleneck (the smallest representation). The most compressed version of the image. It contains global information about the character; at this point, the model has a high-level understanding.

3) Upsampling path (the “rebuild” part). The network increases the image size step by step, $8 \times 8 \rightarrow 16 \times 16 \rightarrow 32 \times 32 \rightarrow 64 \times 64$, using what it learned during compression to reconstruct details.

Skip connections. At every resolution level, the network copies features from the downsampling path and sends them directly to the matching upsampling stage. Why? Because when you compress an image you lose fine detail; skip connections give the network access to the original high-resolution information so it can rebuild sharp results.

Why the U-Net is a good fit for diffusion. The model must take a noisy image and predict the exact noise that was added. That requires (i) understanding global structure (*what character is this?*) and (ii) preserving pixel-level precision (*where exactly is the noise?*). The down path captures overall structure, the up path restores spatial detail, and skip connections prevent loss of fine information.

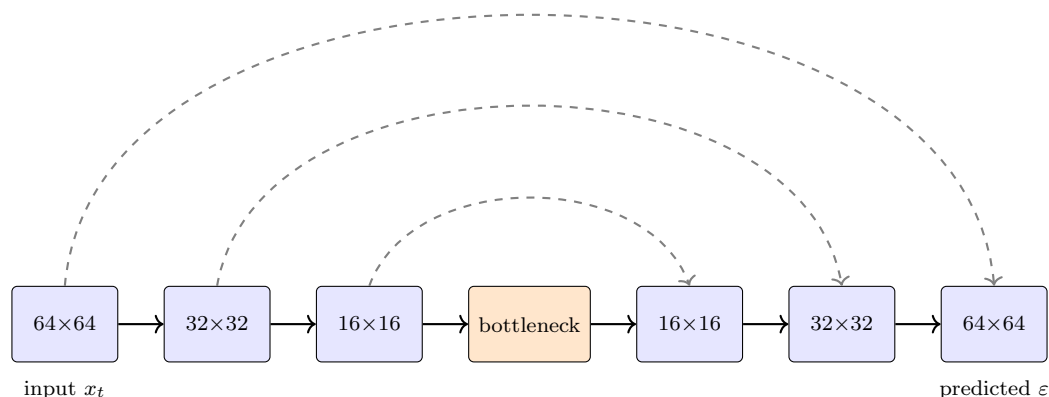


Figure 1: The U-Net at a glance. Down path compresses, the bottleneck holds global context, the up path rebuilds, and skip connections (dashed) ferry fine detail across.

5.3 Constructing the U-Net

```

unet = UNet2DModel(
    sample_size = img_res,
    in_channels = 1,
    out_channels = 1,
    layers_per_block = 2,
    block_out_channels = channel_width,
    down_block_types = ("DownBlock2D",) * len(channel_width),
    up_block_types = ("UpBlock2D",) * len(channel_width),
    num_class_embeds = num_classes + 1, # +1 for the NULL class
).to(device)
unet = torch.nn.parallel.DistributedDataParallel(unet, device_ids=[local_rank])

```

`sample_size`

Spatial resolution the U-Net is built for. Inputs are $B \times 1 \times \text{img_res} \times \text{img_res}$.

`in_channels / out_channels`

EMNIST is greyscale, so 1 in and 1 out (predict noise of the same shape as the input).

`layers_per_block`

How many conv/resnet layers run inside each stage before moving to the next resolution.

`block_out_channels`

Channel widths at each U-Net stage; one entry per resolution level.

`down_block_types / up_block_types`

One block per stage; `DownBlock2D` halves the spatial size each stage, `UpBlock2D` doubles it. (We do *not* use the attention variants — probably overkill for EMNIST; worth testing.)

`num_class_embeds`

Size of the class-embedding table: `num_classes + 1` (the extra slot is the `NULL_CLASS` used for CFG dropout).

`DistributedDataParallel` wraps the model so each process uses its local GPU and gradients are synced across ranks.

5.4 The EMA model

We create a second copy of the U-Net that does *not* train directly. After every optimizer step we update its weights using

$$w_{\text{ema}} \leftarrow \text{EMA_DECAY} \cdot w_{\text{ema}} + (1 - \text{EMA_DECAY}) \cdot w_{\text{current}}.$$

So instead of jumping around like normal SGD updates, the EMA model changes slowly and smoothly over time.

Why? During training, model weights fluctuate from step to step; some updates improve things, some slightly overshoot. The EMA version acts like a smoothed version of the model across many recent training steps. In diffusion models this almost always produces more stable generations, cleaner samples, and better generalisation.

Important. The optimizer updates the main U-Net. The EMA U-Net is updated manually using the moving-average rule, lives in `eval` mode, and has `requires_grad_(False)` on all of its parameters — it is never updated by gradients.

5.5 Scheduler, optimizer, AMP

```
scheduler = DDPMsScheduler(num_train_timesteps=STEPS, beta_schedule=BETA_SCHEDULE)
optimizer = torch.optim.AdamW(unet.parameters(), lr=LEARNING_RATE)
scaler = torch.amp.GradScaler("cuda")
autocast_context = lambda: torch.amp.autocast("cuda", dtype=torch.float16)
```

`DDPMsScheduler`

Defines the forward diffusion process: how many timesteps and how the noise variance β_t grows over time. Used to add noise to clean images during training.

`AdamW`

Standard optimiser for diffusion models.

GradScaler / autocast

When training in `float16`, gradients can underflow. `GradScaler` dynamically scales the loss to keep training stable.

6 The forward diffusion process

In one forward diffusion step we define:

$$\varepsilon_t \sim \mathcal{N}(0, I), \quad \alpha_t = 1 - \beta_t,$$

where β_t is the fraction of *new* noise injected at step t and α_t is the fraction of the *previous* signal retained. Then

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \varepsilon_t.$$

Why we do not loop step-by-step. It may seem strange that each batch contains images at different timesteps. In principle, an image at timestep T has received T additions of Gaussian noise. But T additions of Gaussian noise with mean M and variance V is equivalent to one addition of Gaussian noise with mean $T M$ and variance $T V$. With $M = 0$ and $V = 1$, this simplifies to one addition of Gaussian noise with mean 0 and variance T .

In reality diffusion is slightly more complicated because we do *not* add the same amount of noise at every step. Instead, we use a noise schedule. The default scheduler used here is `squaredcos_cap_v2`, which is a cosine-based schedule. Intuitively, early timesteps add only a little noise, and later timesteps add much more. This is usually better than noising too aggressively at the start — if we destroy the signal too quickly, then later noising steps are less useful.

The closed form. If we repeatedly apply the per-step rule from x_0 to x_t , we get the cumulative form

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha_s,$$

where ε is a single Gaussian noise tensor representing the combined effect of all the individual noise injections $\varepsilon_1, \dots, \varepsilon_t$. This is the key simplification that lets us compute x_t in one shot, and it is exactly what the scheduler does for us.

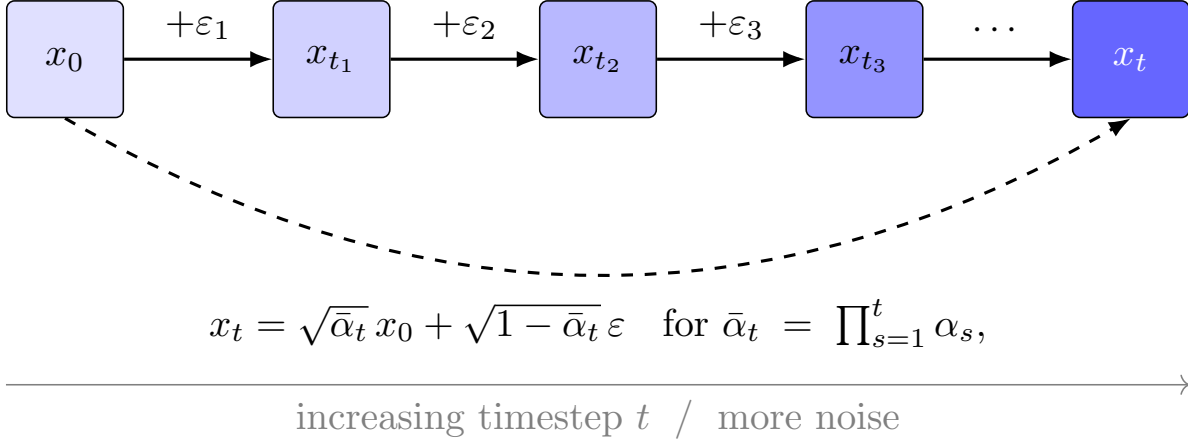


Figure 2: Forward diffusion as a Markov chain (solid arrows) versus the closed-form jump (dashed arrow) that the scheduler actually uses. Each solid step adds a fresh Gaussian noise tensor ϵ_i scaled by the noise schedule; because sums of Gaussians are Gaussian, all t steps collapse into a single draw ϵ scaled by $\sqrt{1 - \bar{\alpha}_t}$.

Reference for *squaredcos_cap_v2*: Nichol & Dhariwal, *Improved Denoising Diffusion Probabilistic Models* (2021), <https://arxiv.org/abs/2102.09672>.

7 Inside the training loop

One epoch is one full pass through the entire dataset. The dataset is divided into mini-batches; one mini-batch $\rightarrow$ one forward pass $\rightarrow$ one loss $\rightarrow$ one optimizer step.

7.1 Setting up the epoch

```
for epoch in range(EPOCHS):
    sampler.set_epoch(epoch) # synchronised reshuffle across GPUs
    t0 = time.time()
    if rank == 0 and tracker is not None:
        tracker.start_epoch()
    losses = []
```

The `DistributedSampler` reshuffles deterministically per epoch so each GPU sees a different slice of the dataset, but in a synchronised way across processes.

7.2 One mini-batch, end to end

The body of the inner loop is conceptually four steps:

1. Apply CFG dropout to the labels.
2. Sample timesteps and noise; build x_t .
3. Predict the noise with the U-Net; compute the MSE loss.
4. Backprop with mixed precision; step the optimizer; update the EMA.

7.2.1 CFG dropout

```
dropout_mask = torch.rand(labels_batch.size(0), device=device) < CFG_DROPOUT_PROB
labels_cfg = torch.where(
    dropout_mask,
    torch.full_like(labels_batch, NULL_CLASS),
    labels_batch,
)
```

`dropout_mask` is a boolean vector like `[False, True, False, False, True, ...]`. Wherever the mask is `True`, the image is trained as if it were a member of the null class. This is what allows us to do conditional/unconditional inference later — it is where our whole diffusion-geometry / force-zoo machinery comes from.

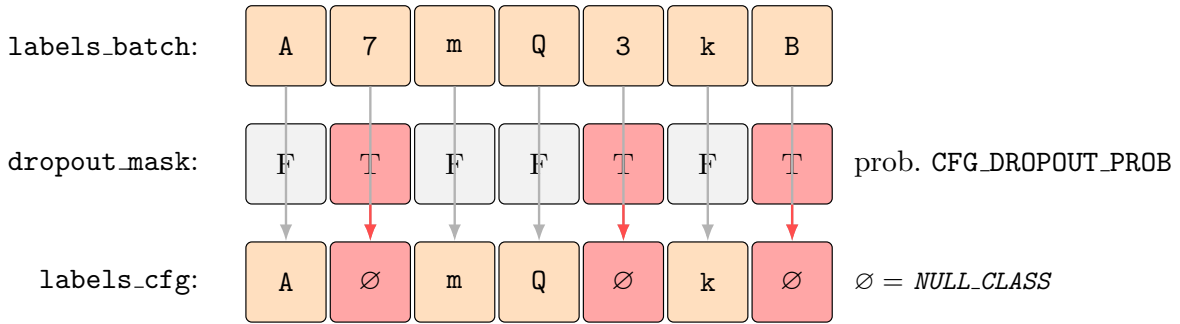


Figure 3: CFG dropout in one mini-batch. With probability `CFG_DROPOUT_PROB` each label is replaced by the null class $\emptyset$. The U-Net therefore sees a mixture of conditional and unconditional examples in every batch, which is what makes classifier-free guidance possible at inference time: the same network can produce both $\varepsilon_\theta(x_t, t, c)$ and $\varepsilon_\theta(x_t, t, \emptyset)$.

7.2.2 Sampling timesteps and noise

```
timesteps_batch = torch.randint(low=0, high=STEPS,
                                size=(images_batch.size(0),), device=device)
noise_batch = torch.randn_like(images_batch)
noisy_images_batch = scheduler.add_noise(
    original_samples = images_batch, # x_0
    noise = noise_batch, # epsilon
    timesteps = timesteps_batch, # t (per image)
)
```

Why random timesteps? The model must learn to denoise images at *every possible noise level*. Looping over all timesteps sequentially would be inefficient, so for each image we sample a timestep uniformly:

$$\mathbb{E}_t [\text{MSE}(\varepsilon_{\text{pred}}, \varepsilon)]$$

is approximated by averaging over many random (t, x_t, ε) triplets. Within a single mini-batch some images are lightly corrupted, some moderately, and some heavily; over many mini-batches this random sampling approximates the full expectation.

7.2.3 Forward pass and loss

```
optimizer.zero_grad(set_to_none=True)
with autocast_context():
    predicted_noise_batch = unet(
        noisy_images_batch,
        timesteps_batch,
        class_labels=labels_cfg,
    ).sample
    loss = F.mse_loss(predicted_noise_batch, noise_batch)
```

The U-Net is trained to solve: “Given x_t (the clean image with t additions of Gaussian noise), can you recover the exact noise ε that was added?”

Why predict noise instead of the clean image directly?

- The noise distribution is simple (Gaussian), making it easier to learn.
- The same objective works across all timesteps.
- Empirically this gives more stable training and better results.

The output `predicted_noise_batch` has the same shape as the input. The MSE is averaged across all pixels, channels, and images in the mini-batch, producing a single scalar that answers “how wrong was the model on this entire mini-batch?”.

7.2.4 Backprop with mixed precision and the optimizer step

```
scaler.scale(loss).backward() # compute gradients (scaled)
scaler.unscale_(optimizer) # unscale for accurate norms / clipping
grad_norm_val = compute_grad_norm(unet.module)
scaler.step(optimizer) # update the live UNet
scaler.update() # adjust scaling factor for next step
```

We trained inside an `autocast float16` context, which improves speed and reduces memory usage but can cause gradients to underflow. `GradScaler`:

1. scales the loss upward before `backward()`,
2. unscales gradients before the optimizer step, and
3. dynamically adjusts its scaling factor over time.

This is what makes mixed-precision training stable.

7.2.5 EMA update

```
with torch.no_grad():
    for ema_param, current_param in zip(ema_unet.parameters(),
                                         unet.module.parameters()):
        ema_param.mul_(EMA_DECAY).add_(current_param, alpha=1 - EMA_DECAY)
```

For each parameter,

$$w_{\text{ema}} \leftarrow \text{EMA_DECAY} \cdot w_{\text{ema}} + (1 - \text{EMA_DECAY}) \cdot w_{\text{current}}.$$

EMA_DECAY is typically close to 1 (e.g. 0.999), so the EMA model changes slowly and represents a long-term average of recent weights. The optimizer updates the live U-Net; the EMA U-Net is never touched by gradients.

7.3 End of epoch: logging and checkpointing

After all mini-batches:

- Compute mean mini-batch loss across the epoch (a rough training signal, not a validation metric).
- Compute elapsed time `dt = time.time() - t0`.
- Only **rank 0** logs and saves; in distributed training every GPU runs this script and we restrict logging/checkpointing to avoid duplication.
- Write an EMA checkpoint to `checkpoints/SIZE_NAME/emnist_MODEL_NAME_epochNNN.pt` containing the EMA `state_dict`, the class list, the scheduler config, image size, channel widths, the variation name, the epoch, the dataset split, beta schedule, EMA decay, CFG dropout probability, the null-class index, and `num_class_embeds`.

8 Post-training: plots, report, clean shutdown

When training finishes (rank 0 only):

1. `generate_all_plots(tracker)` writes plots to `metrics/output/SIZE_NAME/plots`.
2. `generate_report(tracker, config=TRAINING_CONFIG, plot_paths=...)` writes a Markdown report alongside them.
3. Both CSV and JSON metric files (`tracker.csv_path`, `tracker.json_path`) are also retained.

Finally we tear down the distributed group:

```
torch.distributed.destroy_process_group()
```

Summary. `train.py` stitches together a small number of moving parts — `UNet2DModel`, `DDPMScheduler`, `AdamW`, `EMA`, `AMP`, `CFG dropout`, distributed sampling — into a single training loop whose only job is to learn $\epsilon_{\text{pred}} = f(x_t, t, \text{class_label})$. Once that function is learned, the EMA checkpoints become the input to everything else in this repo.